# 2 ЧАСА

# 6. ПРИНЦИПЫ РАБОТЫ С ПОДЗАПРОСАМИ

**Цель практической работы №6**: изучить принципы работы с подзапросами в SQL, а также научиться применять подзапросы для упрощения сложных операций, фильтрации данных и вставки результатов запросов в таблицы.

**Постановка задачи**:

1. Изучить принцип работы и структуру подзапросов
2. Заполнить таблицы, которые были декомпозированы из основной первоначальной таблицы с помощью подзапросов.

## Теоретические основы работы с подзапросами

Подзапросы в SQL представляют собой мощный инструмент для структурирования сложных операций с данными, позволяющий выполнять последовательные и логически связанные запросы в рамках единого выражения. В контексте проектирования баз данных ноутбуков подзапросы становятся особенно ценными при необходимости поэтапного преобразования и переноса данных из денормализованной исходной таблицы в нормализованную схему. Например, операция

*INSERT INTO company (title) SELECT DISTINCT "Company" FROM csv_laptop*

демонстрирует базовый случай использования подзапроса для извлечения уникальных значений и заполнения справочника компаний. Такой подход обеспечивает атомарность операции и гарантирует, что в целевые таблицы будут добавлены только корректные, предварительно отфильтрованные данные, что критически важно для поддержания целостности нормализованной структуры.

Особую значимость для магистрантов представляет использование коррелированных подзапросов и операций соединения при заполнении таблиц, связанных внешними ключами. Как видно из примера заполнения таблицы **model**, подзапросы позволяют устанавливать сложные связи между данными, извлекая идентификаторы из связанных справочников (**model_type**, **company**) на основе значений из исходной таблицы. Этот процесс требует глубокого

понимания как структуры данных, так и семантики связей между сущностями. Более того, использование подзапросов в условиях **WHERE** и **JOIN** позволяет реализовать сложную бизнес-логику и многоуровневую фильтрацию, что особенно актуально при обработке больших объемов данных с высокой степенью нормализации.

С точки зрения производительности, работа с подзапросами требует тщательного анализа планов выполнения запросов, поскольку некорректное использование может привести к существенным временным затратам. Однако в контексте задач миграции и преобразования данных, как в данной практической работе, подзапросы предоставляют незаменимый инструмент для обеспечения согласованности и точности переноса информации.

Для магистранта освоение этого механизма открывает возможности для автоматизации процессов наполнения баз данных, построения сложных ETL-процедур и реализации эффективных методов управления данными в исследовательских проектах и промышленных решениях.

## Выполнение практической работы

1. Заполнение таблицы *company*

```
INSERT INTO company (title)
SELECT DISTINCT "Company"
FROM csv_laptop
WHERE "Company" IS NOT NULL;
```

2. Заполнение таблицы *model_type*

```
INSERT INTO model_type (title)
SELECT DISTINCT "TypeName"
FROM csv_laptop
WHERE "TypeName" IS NOT NULL;
```

3. Заполнение таблицы *laptop_os*

```
INSERT INTO laptop_os (title)
SELECT DISTINCT "OpSys"
FROM csv_laptop
WHERE "OpSys" IS NOT NULL;
```

4. Заполнение таблицы *laptop_cpu*

```
INSERT INTO laptop_cpu (title)
SELECT DISTINCT "Cpu"
FROM csv_laptop
WHERE "Cpu" IS NOT NULL;
```

5. Заполнение таблицы *laptop_gpu*

```
INSERT INTO laptop_gpu (title)
SELECT DISTINCT "Gpu"
FROM csv_laptop
WHERE "Gpu" IS NOT NULL;
```

6. Заполнение таблицы *laptop_rom_type*

Предполагая, что поле "Memory" в таблице csv_laptop содержит тип памяти (например, SSD, HDD):

```
INSERT INTO laptop_rom_type (title)
SELECT DISTINCT "Memory"
FROM csv_laptop
WHERE "Memory" IS NOT NULL;
```

7. Заполнение таблицы *laptop_screen_type*

Если тип экрана определяется разрешением или другими характеристиками, можно использовать "ScreenResolution" или создать отдельное поле для типа. Предположим, что используем "ScreenResolution":

```
INSERT INTO laptop_screen_type (title)
SELECT DISTINCT "ScreenResolution"
FROM csv_laptop
WHERE "ScreenResolution" IS NOT NULL;
```

8. Заполнение таблицы *laptop_rom*

Предполагая, что "Memory" содержит как тип, так и объем памяти, необходимо связать его с таблицей laptop_rom_type. Например:

```
INSERT INTO laptop_rom (lapatop_type_id, volume)
SELECT mrt.id, cl."Memory"
FROM (
```

```
    SELECT DISTINCT "Memory"
    FROM csv_laptop
    WHERE "Memory" IS NOT NULL
) cl
JOIN laptop_rom_type mrt ON mrt.title = cl."Memory";
```

9. Заполнение таблицы *laptop_screen*

Связываем тип экрана с разрешением и размером:

```
INSERT INTO laptop_screen (laptop_screen_type, resolution, inch)
SELECT lst.id, cl."ScreenResolution", cl."Inches"
FROM (
    SELECT DISTINCT "ScreenResolution", "Inches"
    FROM csv_laptop
    WHERE "ScreenResolution" IS NOT NULL AND "Inches" IS NOT NULL
) cl
JOIN laptop_screen_type lst ON lst.title = cl."ScreenResolution";
```

10. Заполнение таблицы *model*

Связываем модель с типом модели и компанией:

```
INSERT INTO model (model_type_id, company_id, title)
SELECT mt.id, c.id, cl."Product"
FROM (
    SELECT DISTINCT "Product", "TypeName", "Company"
    FROM csv_laptop
    WHERE "Product" IS NOT NULL AND "TypeName" IS NOT NULL AND
"Company" IS NOT NULL
) cl
JOIN model_type mt ON mt.title = cl."TypeName"
JOIN company c ON c.title = cl."Company";
```

11. Заполнение основной таблицы *laptop*

После заполнения всех справочных таблиц, можно заполнить основную таблицу laptop, связывая все внешние ключи:

```
-- 11. Заполнение основной таблицы laptop
INSERT INTO laptop (
    model_id,
    laptop_rom_id,
```

```
    laptop_screen_id,
    laptop_os_id,
    laptop_gpu,
    laptop_cpu,
    price_euro,
    weight
)
SELECT
    m.id,
    lr.id,
    ls.id,
    lo.id,
    lg.id,
    lc.id,
    cl."Price_euros",
    cl."Weight"
FROM csv_laptop cl
JOIN model m ON m.title = cl."Product"
JOIN laptop_rom lr ON lr.volume = cl."Memory"
JOIN laptop_screen ls ON ls.resolution = cl."ScreenResolution" AND ls.inch =
cl."Inches"::text
LEFT JOIN laptop_os lo ON lo.title = cl."OpSys"
LEFT JOIN laptop_gpu lg ON lg.title = cl."Gpu"
JOIN laptop_cpu lc ON lc.title = cl."Cpu"
WHERE cl."Product" IS NOT NULL
  AND cl."Product" <> '';
```

Заполненная таблица «laptop» представлена на рис. 39.

| | id | model_id | laptop_rom_id | laptop_screen_id | laptop_os_id | laptop_gpu | laptop_cpu | price_euro | weight |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 18 | 19 | 8 | 55 | 74 | 598 | 3.3kg |
| 2 | 2 | 2 | 26 | 72 | 8 | 5 | 4 | 1260 | 2.7kg |
| 3 | 3 | 3 | 18 | 72 | 8 | 53 | 75 | 613 | 2.2kg |
| 4 | 4 | 4 | 18 | 61 | 8 | 85 | 48 | 348 | 2.1kg |
| 5 | 5 | 4 | 18 | 61 | 4 | 70 | 110 | 298 | 2.1kg |
| 6 | 6 | 5 | 6 | 80 | 8 | 92 | 88 | 799 | 2.4kg |
| 7 | 7 | 6 | 19 | 29 | 8 | 48 | 16 | 1149 | 1.2kg |
| 8 | 8 | 7 | 18 | 61 | 4 | 60 | 74 | 390 | 2.1kg |
| 9 | 9 | 7 | 7 | 61 | 8 | 60 | 74 | 449 | 2.1kg |
| 10 | 10 | 7 | 18 | 61 | 8 | 60 | 74 | 447 | 2.1kg |
| 11 | 11 | 8 | 19 | 71 | 8 | 48 | 71 | 989 | 1.12kg |
| 12 | 12 | 9 | 6 | 70 | 8 | 67 | 4 | 2299 | 4.2kg |
| 13 | 13 | 9 | 6 | 70 | 8 | 14 | 4 | 2599 | 4.2kg |
| 14 | 14 | 10 | 18 | 80 | 4 | 5 | 4 | 779 | 2.4kg |
| 15 | 15 | 10 | 18 | 80 | 4 | 5 | 40 | 798 | 2.4kg |
| 16 | 16 | 10 | 19 | 80 | 4 | 5 | 4 | 879 | 2.5kg |
| 17 | 17 | 10 | 18 | 80 | 4 | 5 | 4 | 846 | 2.4kg |
| 18 | 18 | 11 | 18 | 80 | 8 | 58 | 13 | 530 | 2.2kg |
| 19 | 19 | 12 | 38 | 64 | 8 | 26 | 112 | 485 | 1.4kg |
| 20 | 20 | 12 | 7 | 64 | 8 | 36 | 66 | 435 | 1.4kg |
| 21 | 21 | 13 | 27 | 61 | 4 | 65 | 88 | 441.8 | 2.4kg |
| 22 | 22 | 13 | 27 | 61 | 8 | 60 | 74 | 410.8 | 2.4kg |
| 23 | 23 | 13 | 27 | 61 | 4 | 60 | 57 | 361.8 | 2.4kg |
| 24 | 24 | 14 | 27 | 61 | 8 | 65 | 88 | 619 | 2.23kg |
| 25 | 25 | 15 | 26 | 70 | 8 | 67 | 76 | 1935 | 4.2kg |
| 26 | 26 | 16 | 18 | 80 | 4 | 5 | 40 | 709 | 2.4kg |
| 27 | 27 | 17 | 25 | 61 | 2 | 86 | 67 | 199 | 2.19kg |
| 28 | 28 | 18 | 19 | 73 | 8 | 44 | 53 | 884 | 1.6kg |
| 29 | 29 | 18 | 19 | 73 | 8 | 44 | 75 | 770 | 1.6kg |
| 30 | 30 | 18 | 19 | 73 | 8 | 40 | 88 | 919 | 1.8kg |
| 31 | 31 | 18 | 7 | 73 | 8 | 65 | 5 | 619 | 1.5kg |
| 32 | 32 | 18 | 7 | 73 | 8 | 65 | 5 | 636 | 1.8kg |
| 33 | 33 | 19 | 9 | 64 | 8 | 86 | 61 | 269 | 1.4kg |
| 34 | 34 | 20 | 7 | 60 | 8 | 60 | 11 | 655 | 1.6kg |
| 35 | 35 | 21 | 19 | 61 | 8 | 106 | 56 | 659 | 2.2kg |
| 36 | 36 | 21 | 19 | 70 | 8 | 58 | 13 | 702 | 3kg |

*Рисунок 39. Заполненная таблица «laptop»*

Выполнение практической работы №6 позволило освоить ключевые аспекты применения подзапросов для решения практических задач миграции и преобразования данных в реляционных базах данных. На примере конкретной предметной области (ноутбуки) была продемонстрирована методика поэтапного заполнения нормализованной схемы данных из денормализованной исходной таблицы. Особую ценность представляет опыт использования подзапросов различных типов — от простых конструкций с SELECT DISTINCT для заполнения справочников до сложных запросов с многоуровневыми соединениями таблиц для установления связей через внешние ключи. Этот практический навык необходим для реализации ETL-процессов и обеспечения целостности данных при переходе между различными моделями хранения информации.

Для магистранта приобретенные компетенции работы с подзапросами формируют основу для решения более сложных задач в области управления данными. Умение оптимально структурировать вложенные запросы, понимание различий между коррелированными и некоррелированными подзапросами, а также владение методами оптимизации их производительности составляют навыки современного специалиста по базам данных. Эти знания будут востребованы как в научно-исследовательской деятельности при работе с большими наборами данных, так и в промышленной разработке при создании сложных информационных систем, требующих эффективного манипулирования связанными данными. Освоенные принципы позволяют перейти от простого

использования SQL к профессиональному проектированию сложных запросов, что соответствует уровню подготовки магистра в области ИТ.

## Контрольные вопросы

1. Что такое подзапрос в SQL, и в каких случаях он применяется?
2. Какие виды подзапросов существуют?
3. Как подзапросы используются в разделе WHERE для фильтрации данных?
4. Чем отличаются коррелированные подзапросы от некоррелированных?
5. Как вставить результаты подзапроса в другую таблицу?
6. Можно ли использовать подзапросы в секции FROM, и как это делается?
7. Как работает подзапрос с использованием агрегатных функций (например, MAX, MIN, AVG)?
8. В чем преимущества и недостатки использования подзапросов в SQL?